

IDP-lite Roadmap

A scoped learning project for building a small Internal Developer Platform: service onboarding, CI, deployment, catalogue, observability, and DORA metrics.

Goal

Build a small platform that can onboard a dummy service, give it a standard structure, run CI, deploy it locally or to Kubernetes, show ownership/docs, expose health/metrics, and produce simple DORA-style delivery metrics.

Do not start with Backstage, Terraform, cloud, RBAC, secrets management, or enterprise complexity. Those are later additions after the core flow works.

Recommended build order

#	Step	Output
1	Repo structure	Create the mini-idp repository and folders
2	Dummy service	Use a boring sample API as the workload
3	Standard commands	Add Makefile commands: run, test, lint, build
4	Docker	Containerise the service and verify /health
5	service.yaml	Add ownership and catalogue metadata
6	CI	Run test, lint, and Docker build in GitHub Actions
7	Template	Turn the dummy service into a reusable service template
8	Generator	Build create-service.sh for self-service onboarding
9	Golden path docs	Document how teams create, run, test, and deploy services
10	Catalogue	Generate a service catalogue from service.yaml files
11	Local deploy	Run services through Docker Compose
12	Kubernetes	Deploy to Kind or Minikube
13	Helm	Package deployment as a reusable chart
14	Health checking	Check running services from metadata
15	Observability	Add metrics, logs, Prometheus, and Grafana basics
16	Deployment tracking	Record deployments in a CSV or simple database
17	DORA report	Calculate deployment frequency, lead time, CFR, MTTR
18	Docs site	Publish the guidance, catalogue, and reports
19	Project story	Write up problem, design, trade-offs, and future improvements

Core foundation phases

Phase 0 - Set up the repo

Create the skeleton for the platform project.

```
mini-idp/  
README.md  
docs/  
templates/  
services/  
platform-cli/  
catalogue/  
deploy/  
metrics/
```

Done: Done when the repo exists with the folders above and a README that explains the IDP-lite goal.

Phase 1 - Choose a dummy service

Use a sample API. Do not spend much time building application logic.

```
Minimum endpoints:  
GET /health  
GET /version  
GET /metrics
```

Done: Done when services/dummy-api can run locally and /health works.

Phase 2 - Standard developer commands

Add a Makefile so every service has predictable commands.

```
make run  
make test  
make lint  
make build  
make docker-run
```

Done: Done when a new developer can run the service without reading lots of docs.

Phase 3 - Containerise the service

Add Dockerfile and docker-compose.yml so the service runs consistently.

```
docker build -t dummy-api .  
docker run -p 8000:8000 dummy-api  
# or  
docker compose up
```

Done: Done when the service runs in a container and /health works.

Phase 4 - Add service metadata

Add service.yaml to describe ownership, type, language, repo, healthcheck, and runbook.

```
name: dummy-api  
owner: platform-learning  
type: backend  
language: python  
repo: local  
healthcheck: /health  
runbook: README.md  
tier: learning
```

Done: Done when every service has a consistent metadata file.

Phase 5 - Add CI

Add a GitHub Actions workflow for a standard quality gate.

```
.github/workflows/ci.yml
```

```
Pipeline:  
- install dependencies  
- run tests  
- run lint  
- build Docker image
```

Done: Done when every push or PR runs test, lint, and image build.

Phase 6 - Turn the dummy service into a template

Move or copy the service into templates/basic-api and replace values with placeholders.

```
{{SERVICE_NAME}}  
{{OWNER}}  
{{PORT}}
```

Done: Done when the template can be copied and renamed into a new service.

Phase 7 - Build the service generator

Create a small self-service onboarding script.

```
./platform-cli/create-service.sh orders-api
```

```
Creates:  
services/orders-api/  
README.md  
Makefile  
Dockerfile  
docker-compose.yml  
service.yaml  
.github/workflows/ci.yml  
app/  
tests/
```

Done: Done when create-service.sh generates a service that can run and pass tests.

Platform capability phases

Phase 8 - Write the golden path docs

Create docs/golden-path.md, docs/getting-started.md, and docs/service-standards.md. Explain how to create, run, test, build, deploy, and own a service.

Done: Done when someone else could follow your docs without asking you.

Phase 9 - Build a basic service catalogue

Create catalogue/generate_catalogue.py. It scans services/*/service.yaml and generates catalogue/catalogue.md.

Done: Done when make catalogue shows services, owner, type, language, healthcheck, and runbook.

Phase 10 - Add local deployment

Create deploy/docker-compose.platform.yml to run one or more generated services.

Done: Done when make deploy-local starts the service and /health responds.

Phase 11 - Add Kubernetes deployment

Use Kind or Minikube. Add deployment.yaml and service.yaml, then wrap with make deploy-k8s and make undeploy-k8s.

Done: Done when the dummy service runs in a local Kubernetes cluster.

Phase 12 - Add Helm

Create deploy/charts/basic-service with Chart.yaml, values.yaml, and templates for deployment and service.

Done: Done when a generated service can be deployed using a reusable Helm chart.

Phase 13 - Add health checking

Create platform-cli/check-health.sh. It reads service.yaml, calls health endpoints, and prints service status.

Done: Done when make health checks all running services.

Phase 14 - Add basic observability

Start with /health, /version, /metrics, and structured logs. Then add Prometheus and Grafana.

Done: Done when Prometheus can scrape the service and Grafana has one basic dashboard.

Example catalogue output

Service	Owner	Type	Language	Healthcheck	Runbook
dummy-api	platform-learning	backend	python	/health	README
orders-api	platform-learning	backend	python	/health	README

Metrics and reporting phases

Phase 15 - Add deployment event tracking

Create metrics/deployments.csv to record deployments. Start simple before using a database.

```
service_name,version,deployed_at,status,recovered_at,commit_sha,pr_merged_at
orders-api,1.0.0,2026-07-06T12:00:00,success,,abc123,2026-07-06T10:30:00
orders-api,1.0.1,2026-07-07T14:00:00,failed,2026-07-07T15:20:00,def456,2026-07-07T12:15:00
```

Done: Every deployment can be recorded.

Phase 16 - Add simple DORA metrics

Create metrics/dora_report.py and generate metrics/dora_report.md.

Metric	Simple calculation
Deployment frequency	Count deployments per service
Lead time for changes	deployed_at minus pr_merged_at
Change failure rate	failed deployments divided by total deployments
MTTR	recovered_at minus deployed_at for failed deployments

Done: make dora outputs a basic DORA report.

Phase 17 - Add a simple portal/docs site

Start with Markdown. Then consider MkDocs. Backstage should come later, after the platform concepts are clear.

- Getting started
- Golden path
- Service catalogue
- Deployment guide
- Observability guide
- DORA report
- Runbooks

Done: One site or page shows the platform guidance and catalogue.

Phase 18 - Polish the project story

Create docs/project-story.md covering the problem, platform goal, golden path, architecture, trade-offs, deliberate exclusions, and future improvements.

Use this summary: *I built an IDP-lite to understand platform engineering by creating a golden path for service onboarding, CI, deployment, observability, catalogue metadata, and DORA reporting.*

What not to build yet

Avoid these until the core project works. They are real IDP topics, but they will slow you down too early.

- Backstage
- Terraform
- AWS/GCP/Azure
- Authentication and RBAC
- Secrets management
- Multi-team permissions
- Microservices
- Service mesh
- Complex GitOps
- Production-grade monitoring

First 5 concrete tasks

#	Task	Output
1	Create GitHub repo	mini-idp
2	Add folders	services, templates, platform-cli, docs, catalogue, deploy, metrics
3	Add or copy a dummy API	services/dummy-api
4	Add Makefile	run, test, lint, build, docker-run
5	Add Dockerfile	/health works in Docker

Success criteria

- A dummy service can be onboarded through your platform flow.
- A generated service has standard docs, tests, Docker, CI, metadata, and run commands.
- The service can be deployed locally and later to Kubernetes using Helm.
- A catalogue can list services and ownership from service.yaml files.
- Basic health, metrics, and DORA reports are visible.
- The README and project story explain the platform engineering thinking, not just the tools.